

Tres formas principales de crear y trabajar con objetos en **JavaScript**:

1. Objeto literal
2. Constructor de objetos (función constructora)
3. Clases (introducidas en ES6)

1. Objeto Literal

La forma más sencilla y directa de crear un objeto en JavaScript.

```
// Creamos un objeto literal llamado 'persona'
const persona = {
  // Propiedades
  nombre: "Ana",
  edad: 25,
  profesion: "Desarrolladora",

  // Método (función dentro del objeto)
  saludar: function() {
    console.log(`Hola, mi nombre es ${this.nombre} y soy ${this.profesion}.`);
  }
};

// Accedemos a las propiedades
console.log(persona.nombre); // Muestra: Ana
console.log(persona.edad);   // Muestra: 25

// Llamamos al método
persona.saludar(); // Muestra: Hola, mi nombre ↓ Ana y soy Desarrolladora.
```

● Explicación:

- Usamos llaves `{ }` para definir el objeto directamente.
- Dentro declaramos **propiedades** (pares clave-valor).
- Los **métodos** son funciones asociadas al objeto.
- `this` hace referencia al propio objeto.



2. Constructor de Objetos (Función Constructora)

Antes de que existieran las clases, esta era la forma tradicional de crear múltiples objetos del mismo tipo.

```
// Definimos una función constructora para crear objetos 'Coche'
function Coche(marca, modelo, anio) {
  // 'this' hace referencia al nuevo objeto que se creará
  this.marca = marca;
  this.modelo = modelo;
  this.anio = anio;

  // Método del objeto
  this.mostrarInfo = function() {
    console.log(`Coche: ${this.marca} ${this.modelo} (${this.anio})`);
  };
}

// Creamos instancias (objetos) usando 'new'
const coche1 = new Coche("Toyota", "Corolla", 2020);
const coche2 = new Coche("Ford", "Focus", 2022);

// Llamamos a los métodos
coche1.mostrarInfo(); // Muestra: Coche: Toyota Corolla (2020)
coche2.mostrarInfo(); // Muestra: Coche: Ford Focus (2022)
```

● Explicación:

- Una **función constructora** comienza con mayúscula por convención.
- Al usar **new**, se crea un nuevo objeto y **this** apunta a él.
- Es útil para crear múltiples instancias con la misma estructura.
- Cada instancia tiene sus propios datos.

3. Clases (ES6+)

A partir de ECMAScript 2015 (ES6), se introdujo la sintaxis de **clases**, que simplifica y hace más legible la creación de objetos y herencia.

```
// Definimos una clase 'Animal'
class Animal {
  // El constructor define las propiedades del objeto
  constructor(nombre, tipo) {
    this.nombre = nombre;
    this.tipo = tipo;
  }

  // Método de la clase
  hacerSonido() {
    console.log(`${this.nombre} hace un sonido.`);
  }
}

// Creamos una subclase que hereda de 'Animal'
class Perro extends Animal {
  constructor(nombre, raza) {
    // Llamamos al constructor de la clase padre con 'super'
    super(nombre, "Perro");
    this.raza = raza;
  }
}
```

```
// Sobrescribimos un método
hacerSonido() {
  console.log(`${this.nombre} (un ${this.raza}) ladra: ¡Guau!`);
}

// Creamos instancias
const animal1 = new Animal("Toby", "Gato");
const perro1 = new Perro("Max", "Labrador");

// Usamos los métodos
animal1.hacerSonido(); // Muestra: Toby hace un sonido.
perro1.hacerSonido(); // Muestra: Max (un Labrador) ladra: ¡Guau!
```

Explicación:

- Las clases son una **forma moderna** de crear objetos con constructor y métodos.
- **constructor()** se ejecuta al crear el objeto.
- **extends** permite **heredar** propiedades y métodos de otra clase.
- **super()** llama al constructor de la clase padre.

- Permite definir **herencia** y **reutilizar código** fácilmente.

Resumen de diferencias

Tipo de objeto	Sintaxis	Ideal para	Herencia
Objeto literal	<code>{}</code>	Objetos únicos o simples	 No
Función constructora	<code>function Nombre() {}</code>	Múltiples objetos similares	 Con prototipos
Clase (ES6)	<code>class Nombre {} </code>	Código más claro y orientado a objetos	 Sí